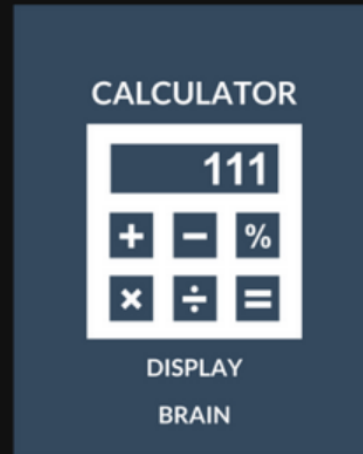


Object Oriented Programming

Created by <Academia de Código_>

Composition

Design technique to implement HAS-A relationship in classes



The calculator HAS-A Brain and HAS-A Display.

Let's create the Brain

Will be responsible for performing all the math calculator operations

```
/**
 * Brain.java
 */

public class Brain {

    public int getResult(Operation operation, int number1, int number2) {

        // ...

    }

}
```

Let's create the Display

The calculator Display will be responsible for showing all the output.

```
/**
 * Display.java
 */

// Create a new class
public class Display {

    public void showResult(int result) {
        System.out.println(result);
    }
}
```

The Calculator has-a Brain and a Display

```
/**
 * Calculator.java
 */

public class Calculator {

    // ...

    private Brain brain;    // responsible for performing the math
    private Display display; // responsible for showing results

    public Calculator(String brand, String color) {

        // ...

        brain = new Brain();
        display = new Display();
    }
}
```

Make use of the new classes

```
/**
 * Calculator.java
 */

// perform chosen operation on provided numbers
public void executeOperation(Operation operation, int number1, int number2) {

    // ask the brain to compute the result
    int result = brain.getResult(operation, number1, number2);

    // ask the display to show the result
    display.showResult(result);
}
```

Composition and Delegation

Design Pattern where an object exposes behaviour to the outside

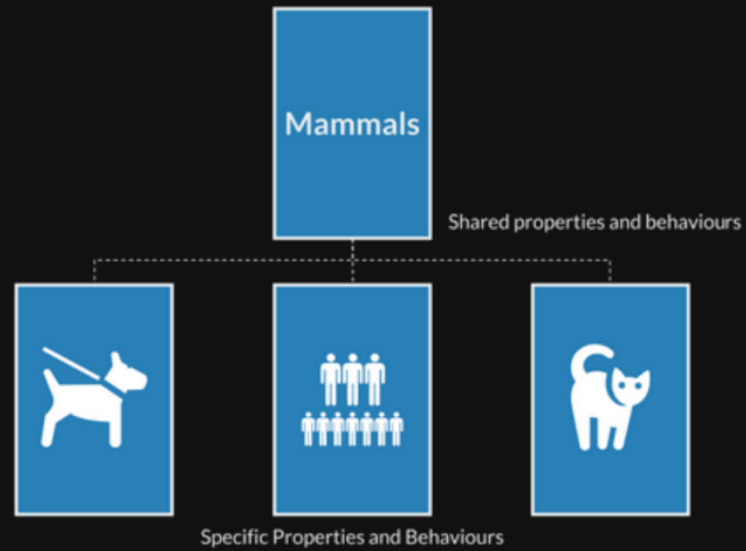
Object delegates the responsibility for implementing the behaviour
to a composed object

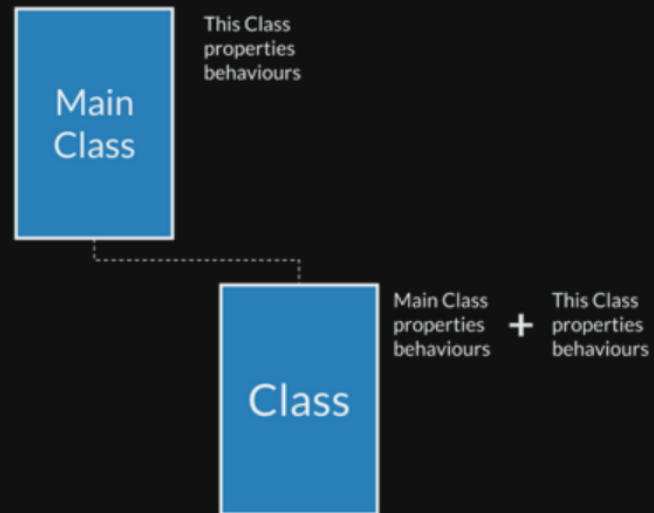
{ Exercise }

Hotel Simulation using Composition and Delegation

Inheritance

Inheritance is the design technique to implement IS-A relationship in classes. It allows classes to be derived from other classes, thereby inheriting their fields and methods.





Why inheritance?

Allows you to reuse the fields and methods of the existing class without having to write them again (easier to use and modify).

Calculator



Properties
brand
color

Behaviours
add
subtract
divide
multiply

Calculator



Properties
brand
color

Behaviours
add
subtract
divide
multiply

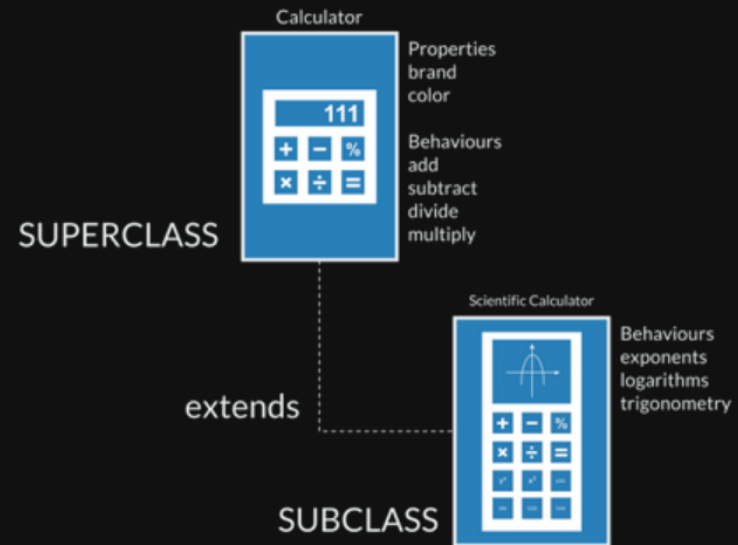
Scientific Calculator



Properties
brand
color

Behaviours
add
subtract
divide
multiply
exponents
logarithms
trigonometry

The Scientific Calculator IS-A Calculator.



The Scientific Calculator can do everything a Calculator does and more.

Use Inheritance when

- The subclass is a more specific type of the superclass;
- Share behaviours among classes of the same general type;
- The subclass and the superclass pass the IS-A test.

Do not use Inheritance

- Reuse code from another class without existing a proper relationship among the classes;
- The subclass and the superclass do not pass the IS-A test;
- The relationship is better described using the HAS-A test.

Inheritance vs Composition

Composition and delegation can be used as an alternative to inheritance.

Inheritance is a good strategy when a close relationship exists in
between parent and child object

Inheritance couples objects very close

Composition and Delegation is the more flexible way to express a relationship between classes

Extending the Calculator Class

```
/**
 * ScientificCalculator.java
 */

// The Scientific Calculator IS-A Calculator
public class ScientificCalculator extends Calculator {
}
```

Adding Methods

```
/**
 * ScientificCalculator.java
 */

// The Scientific Calculator IS-A Calculator
public class ScientificCalculator extends Calculator {

    // Returns the value of the first argument
    // raised to the power of the second argument
    public double pow(double number, double power) {
        return Math.pow(number, power);
    }
}
```

Method Overriding

Declaring a method in a subclass which is already present in a parent class is known as method overriding

```
/**
 * ScientificCalculator.java
 */
// The Scientific Calculator IS-A Calculator
public class ScientificCalculator extends Calculator {

    @Override
    public void powerOn() {
        super.powerOn(); // call the power method on the superclass
        display.showMessage("And I can also do some more advanced operations such as powers");
    }
}
```

Default Constructor

The Java compiler will insert a default constructor on a class if no constructor is found

```
// Calculator Class
public class Calculator {

    // default constructor contains no arguments and an empty body
    public Calculator() {}

}
```

```
// ScientificCalculator class
public class ScientificCalculator extends Calculator {

    // default constructor includes a call to super if inheriting
    public ScientificCalculator() {
        super();
    }

}
```

Non-default Constructor

The superclass default constructor is always implicitly called from any subclass constructor

```
public class ScientificCalculator extends Calculator {  
    private String brand;  
    public ScientificCalculator() {  
        super(); // implicit constructor call will be added by compiler if not present  
        this.brand = brand;  
    }  
}
```

No Default Constructor

What if the super class has no default constructor?

```
public class Calculator {  
    private String brand;  
  
    public Calculator(String brand) {  
        this.brand = brand;  
    }  
}  
  
public class ScientificCalculator extends Calculator {}
```

Program will not compile, default subclass constructor calls non existing super class default constructor

```
public class ScientificCalculator extends Calculator {  
    public ScientificCalculator(String brand) {  
        super(brand); // superclass has no default constructor, explicit call to super is required  
    }  
}
```

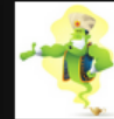


Magic Lamp



- Releases a new genie every time it is rubbed
- When enchanted (instantiated), the maximum number of genies is set
- Genies can be Friendly (even) or Grumpy (odd)
- When the number of genies is exhausted, it releases a recyclable demon
- Has the ability to recharge itself by recycling a demon
- We can compare two lamps by their capacity, number of remaining genies and number of times it has been recharged

Friendly Genie



- When released from the lamp (instantiated), the maximum number of wishes is set
- It can only grant one wish at a time
- It can grant wishes if the maximum has not been reached

Grumpy Genie



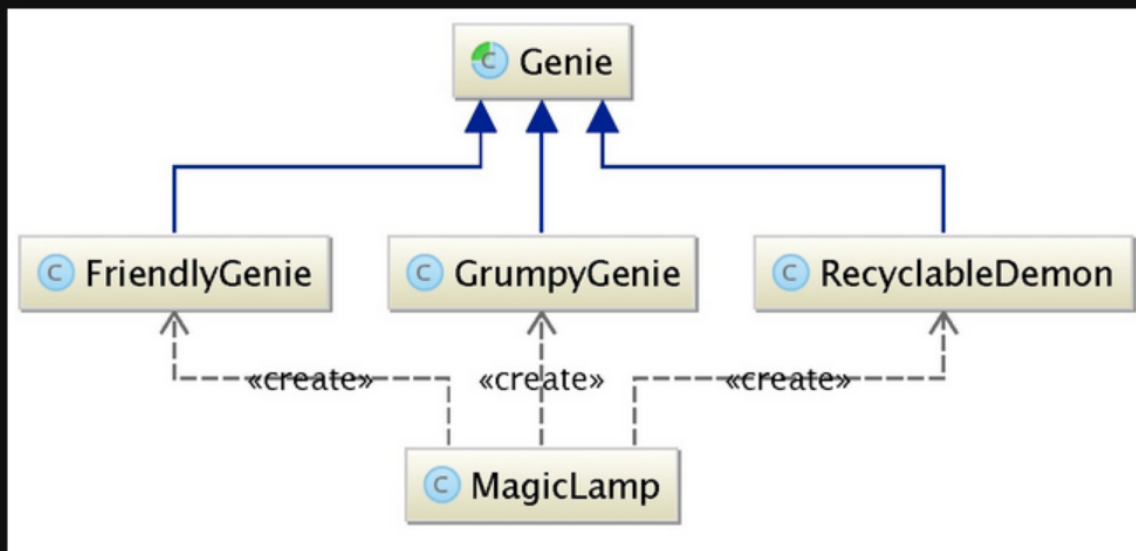
- When released from the lamp (instantiated), the maximum number of wishes is set
- It can only grant one wish at a time
- Grants ONLY ONE wish, even if the maximum has not been reached

Recyclable Demon



- When released from the lamp (instantiated), the maximum number of wishes is set
- It can only grant one wish at a time
- Grants ALL wishes, even if the maximum has already been reached
- It can recharge a magic lamp, if recycled
- It will not grant any more wishes after being recycled
- It can only be recycled once

UML Class Diagram



Polymorphism

polýs (many) + morphḗ (form): The same behaviour can be assumed differently by objects of the same type.



Polymorphism

Feature that allows for the decision of which method to execute to be delayed to runtime.

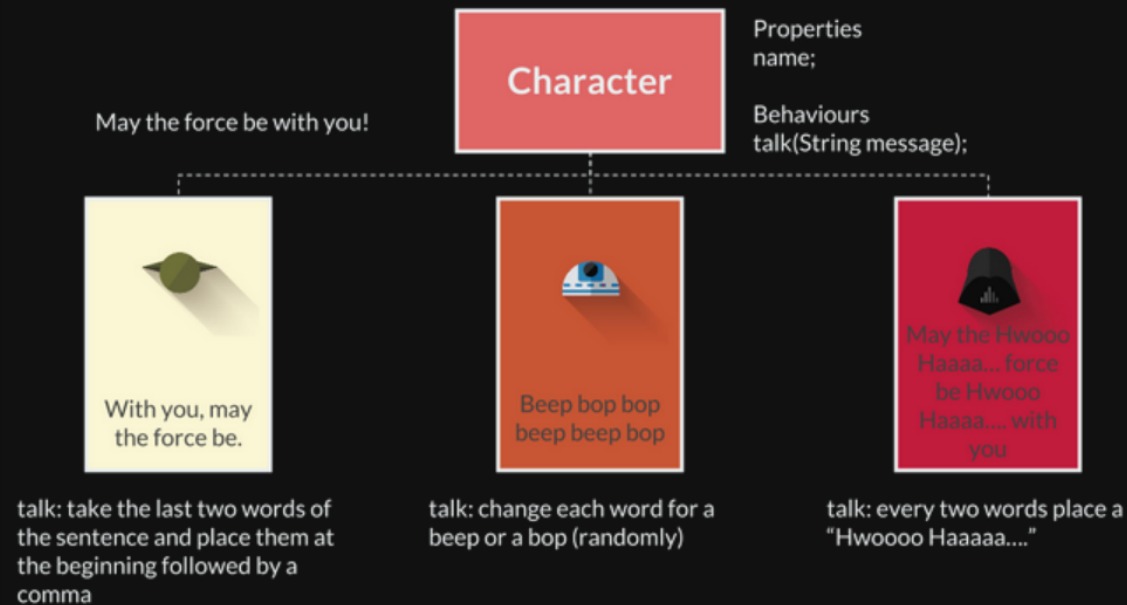
This decision will depend on which instance of a set of classes of the same type the method was called on.

Wait, how?...

Polymorphism is enabled by a couple of inheritance features, namely:

- the ability to override methods of a superclass in a subclass; and
- the ability of a superclass reference variable to refer to a subclass object.





Get them all talking

```
/**
 * Main.java
 */
public class Main {

    public static void main(String[] args) {

        Yoda yoda = new Yoda();
        R2D2 r2d2 = new R2D2();
        DarthVader darthVader = new DarthVader();

        // we can assign subclass object types to an array of superclass objects
        Character[] characters = { yoda, r2d2, darthVader };
    }
}
```

Get them all talking

```
/**
 * Main.java
 */
public class Main {

    public static void main(String[] args) {

        Yoda yoda = new Yoda();
        R2D2 r2d2 = new R2D2();
        DarthVader darthVader = new DarthVader();

        Character[] characters = { yoda, r2d2, darthVader };

        for(int i = 0; i < characters.length; i++) {

            // at compile time we don't know which method will be called
            // the method on the right class (super or sub) will be called at runtime
            characters[i].talk(message);
        }
    }
}
```

Why Polymorphism?

- Write code that will work without knowing specifically which subclass instance will be chosen
- Possibility to add more subclasses without breaking the existing code

Abstract Classes

Conceptual classes that cannot be instantiated, but can be subclassed

Why Abstract Classes?

Providing a base skeleton with properties and methods with the sole purpose of being extended

Character Class

```
/**
 * Character.java
 */
public abstract class Character {
    private String name;

    public abstract void talk(String message);
}
```

There is no point in creating Character object instances.
The class must be extended.

Abstract methods

Methods that ensure the existence of a specific behaviour without declaring an implementation

A class with an abstract method must be abstract as well

Declaring an Abstract method

```
/**
 * Character.java
 */
public abstract class Character {

    private String name;

    // I want my characters to talk but I have no idea how each one will do it
    public abstract void talk(String message);

}
```

An abstract method has no body.
The method must be overridden.

Overriding the Abstract method

```
/**
 * Yoda.java
 */

public class Yoda extends Character {

    @Override
    public void talk(String message) {
        System.out.println(message);
    }
}
```

A subclass must implement all the abstract methods, otherwise it must be abstract as well.

The Object Class

- Every class, in Java, extends class Object;
- When you don't explicitly extend another class, you are implicitly extending the Object class.

Object Methods

- equals(Object o);
- getClass();
- toString();
- hashCode();

Object instanceof

```
/**
 * Main.java
 */

public static void main(String[] args) {

    // saving the yoda as an object
    Object o = new Yoda();

    // the compiler checks the class of the reference variable and not the actual object
    // o.talk("hello");

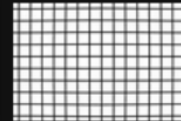
    // we need to cast back to the original type in order to use its methods
    if(o instanceof Yoda) {
        Yoda yoda = (Yoda)o;
        yoda.talk("The force is strong in this one");
    }
}
```

Car Crash



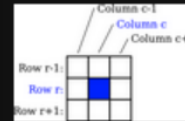
Field

- Grid for placing cars
- Knows how to draw a bunch of cars
- Usage of the lanterna lib for writing to screen



Position

- Describes where a Car lives on the Grid
- Updating position results in moving the Car



Cars

- Different types of cars exist
- Cars move at different speeds and in different ways
- A car can crash at any time



Game

- Creates cars
- Makes all the cars move
- Checks for collisions between cars



UML Class Diagram

